Constructive Logic: Heyting Semantics for Digital Composition

Constructive logic provides a semantics for digital composition in which proofs are not merely external justifications but internal witnesses of implementability. In this chapter, we interpret circuit specifications constructively, emphasizing the distinction between what can be proved to exist and what can be exhibited as an explicit artifact. This perspective is especially natural for digital design, where a specification should ideally correspond to a realizable implementation and where failure modes can be analyzed as counterexamples rather than as abstract negations.

A useful operational summary is given by the quantity $\varepsilon(r)$, which we interpret as a comparison between proof obligations and counterexample rate. Informally, a low value of $\varepsilon(r)$ indicates that the design space is dominated by obligations that can be discharged constructively, while a high value indicates that counterexamples are frequent relative to the claims being made. In a build-oriented workflow, $\varepsilon(r)$ can be used as a chapter-specific diagnostic: it does not replace correctness, but it helps track whether a proposed specification is becoming more amenable to constructive realization.

The semantic setting for this chapter is provided by Heyting algebras rather than Boolean algebras. A Boolean algebra validates classical principles such as excluded middle in full generality, whereas a Heyting algebra models intuitionistic logic, where negation is understood as implication into falsity and where double negation need not collapse to the original proposition. For digital composition, this distinction matters because many design claims are naturally constructive: one may want an explicit witness for a signal assignment, a decomposition, or a refinement map, rather than a nonconstructive existence argument.

In this setting, logical connectives correspond to algebraic operations that preserve constructive meaning. Conjunction models simultaneous satisfaction of specifications, disjunction models a choice between alternatives, and implication models refinement or transformation of one specification into another. Closure under constructive connectives is therefore a central theme: if a family of circuit specifications is closed under the relevant intuitionistic operations, then composite designs can be reasoned about internally without leaving the constructive framework. By contrast, classical reasoning via unrestricted double negation can obscure implementation content. The double-negation caveat is that a statement of the form $\neg\neg P$ does not, in general, provide a witness for P; in circuit terms, a proof that a behavior cannot be refuted is not the same as a construction of the behavior itself.

The contrast between Heyting and Boolean semantics becomes especially visible in the treatment of exclusive-or. Classically, XOR is often introduced as a derived connective with a simple truth table. Constructively, however, one must be careful about how the exclusive-or specification is presented, because the intended meaning is not merely that exactly one of two propositions holds in an abstract sense, but that one can determine which side holds and provide the corresponding witness. This makes constructive XOR a useful test case for the difference between Boolean semantics and Heyting semantics in digital composition.

The Curry–Howard correspondence supplies the bridge from logic to circuits. Under this interpretation, types are specifications and terms are implementations. A proposition corresponds to a type, a proof corresponds to a program, and a proof term can be read as a circuit or circuit fragment that realizes the specification. This viewpoint is particularly effective for compositional hardware design: if a type expresses the interface contract of a module, then a term inhabiting that type is an executable witness that the module satisfies its contract. Composition of proofs becomes composition of circuits, and logical elimination rules become wiring principles.

From this perspective, constructive logic is not merely a philosophical preference; it is a discipline for artifact generation. A proof that a circuit exists should ideally be accompanied by the

circuit itself, or at least by a machine-checkable derivation from which the circuit can be extracted. This is the motivation behind the use of proof assistants such as Agda and Coq. In such systems, one can formalize a specification, prove that it is inhabited, and then extract executable code or circuit descriptions from the proof term. The resulting artifact is not an informal sketch but a buildable object that can be checked, compiled, and integrated into a larger design flow.

The artifact-oriented workflow suggested here has three layers. First, one states a constructive specification in a type-theoretic or intuitionistic logical language. Second, one proves the specification using only constructive principles, thereby ensuring that the proof carries computational content. Third, one extracts or translates the proof into an implementation, such as a circuit netlist, a hardware description, or a verified functional model. In this way, the logical semantics and the engineering semantics remain aligned.

A practical consequence of this alignment is that the semantics of failure also becomes constructive. When a specification is not realizable, the relevant information is often a counterexample, a refutation, or a witness of inconsistency in the assumed constraints. Thus the chapter's emphasis on $\varepsilon(r)$ can be read as a way of balancing proof obligations against counterexample rate: the more the design is expressed constructively, the more directly one can distinguish between a missing proof and a genuine impossibility.

The chapter therefore sets up a semantic framework for later constructions in the book. It complements the earlier algebraic and compositional foundations by showing how logical structure can be used to certify digital artifacts, and it prepares the ground for subsequent case studies in which gates, modules, and larger circuits are derived from specifications rather than assembled ad hoc. The central message is that constructive logic, Heyting semantics, and Curry–Howard interpretation together provide a principled route from specification to implementation, with proofs serving as build scripts and extracted circuits serving as executable evidence.